

Explaining Your JWT Auth Project — Interview Guide

Diagrams

Diagram 1 — the outer request pipeline. Every request, protected or not, passes through this sequence.

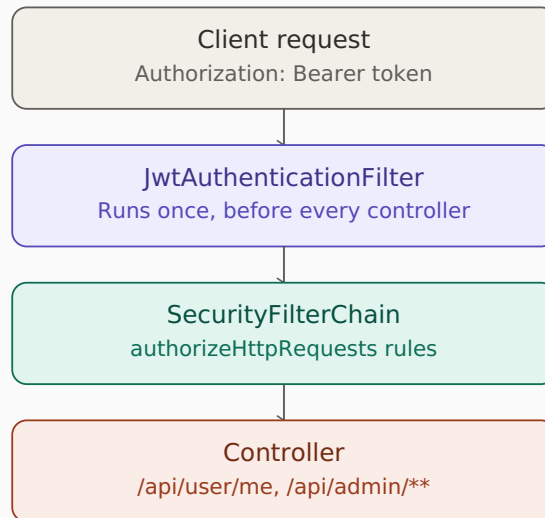
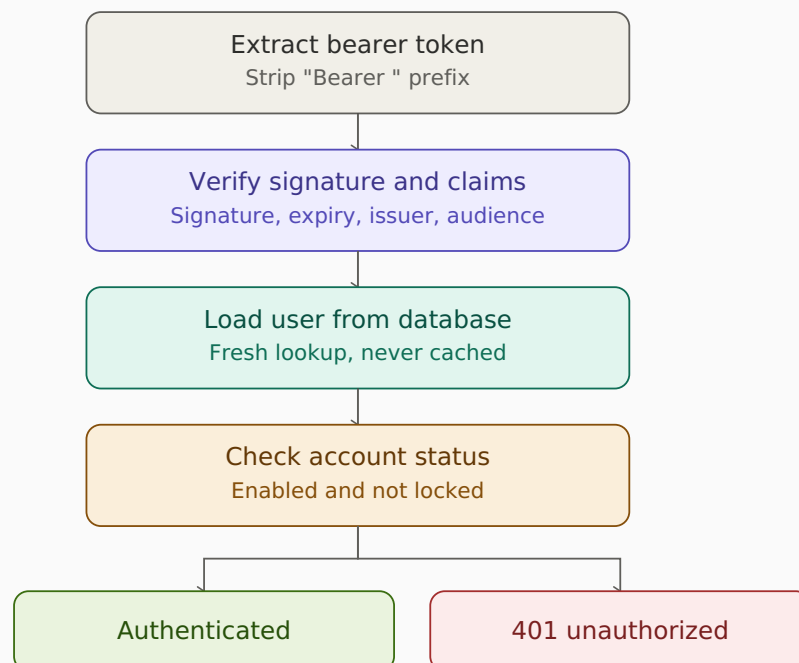


Diagram 2 — what happens inside the filter when a token is validated. Any failure at any step routes to the same 401 branch.



Explaining Your JWT Auth Project — Interview Guide

Use this to rehearse OUT LOUD, not just read silently. Record yourself once against the 90-second version below and listen back — that's the single highest-leverage thing you can do with this document.

The 90-second version (say this first, unprompted)

"I built a Spring Boot authentication service using Spring Security 6's current filter-chain model — not the old `WebSecurityConfigurerAdapter`, which was removed in Security 6. The core design decision was splitting access and refresh tokens: access tokens are short-lived, stateless JWTs verified by signature alone, but refresh tokens are opaque random strings persisted in the database with a revoked flag. That's deliberate — a self-contained JWT can't be revoked before it expires, which is unacceptable for a credential that lives seven days. Every refresh also rotates the token, so a stolen refresh token is only useful once.

I also hardened it against a cross-service token confusion issue — if two services accidentally share the same signing secret, a token from Service A could get accepted by Service B. I fixed that by adding issuer and audience claims that each service checks against its own name."

That's your anchor. Everything below is what you say when they dig into any piece of it.

Part 1 — Walk the request flow (use the diagrams as your mental model)

Say this while sketching or referencing the flow:

"Every request passes through `JwtAuthenticationFilter` before it reaches Spring's own authentication filter — registered via `addFilterBefore`. The filter extracts the Bearer token, and if it's missing or invalid, it doesn't reject the request itself — it just leaves the `SecurityContext` empty and lets `authorizeHttpRequests` and the `AuthenticationEntryPoint` decide the response. That's a single-responsibility split: the filter's only job is 'is there a valid identity here', not 'is this endpoint allowed for this identity'."

If asked "why not reject directly in the filter":

"Because then the filter would need to know about routing rules — which paths are public, which need which role — and that logic already lives in `authorizeHttpRequests`. Duplicating it in the filter would mean two places that can drift out of sync."

Part 2 — The four validation checks (this is your strongest material)

Walk through each step and, critically, say **why**, not just **what**:

1. **Extract token** — strip `Bearer`.
2. **Verify signature + issuer + audience** — "Signature proves it wasn't tampered with. Issuer and audience prove it was actually meant for *this* service — that's the fix for the cross-service confusion I mentioned."
3. **Load user from the database — every request, not cached.** "This is the one people miss. If I trusted the token's claims alone, a user disabled or deleted after the token was issued would keep working until natural expiry. Re-fetching from the DB means the current account state always wins over what's baked into the token."
4. **Check enabled / not locked.** "Even after re-fetching, I initially forgot to actually use that fetched status — the DB lookup happened, but nothing enforced it. Fixed that by checking `isEnabled()`, `isAccountNonLocked()`, `isCredentialsNonExpired()` inside `isTokenValid`."

This last point is your best story — see Part 4.

Part 3 — Access vs refresh tokens (expect deep probing here)

Question	Your answer
"Why not just use one long-lived JWT?"	"Then I could never revoke it early — no logout, no 'log out other devices', no response to a compromised token, short of waiting out its lifetime."
"Why not make the refresh token a JWT too?"	"Same problem, just pushed to a longer window. The refresh token needs to be revocable, so it has to be a lookup key against something I control — a DB row — not a self-contained credential."
"What stops someone reusing a stolen refresh token?"	"Rotation — every refresh call revokes the old token and issues a new one. If an attacker and the real user both hold the same refresh token, whoever uses it first invalidates it for the other, which at minimum signals the compromise."
"What if the DB is down — does auth fail?"	"Access-token validation doesn't need the DB for signature checking, but it does need it for the user re-fetch, so yes — an outage affects auth. That's a real tradeoff of the 'always re-check the DB' design versus a purely stateless approach. Worth mentioning as an explicit tradeoff, not a gap you missed."

Part 4 — Your best STAR story (the account-status gap)

This is genuinely good material because it's a *real bug you found and fixed*, not a rehearsed textbook answer. Use the STAR shape:

Situation: "While building JWT auth, I had `isTokenValid` checking signature, expiry, and username match." **Task:** "I needed to make sure a disabled or banned account couldn't keep using an already-issued token." **Action:** "I noticed the code was already fetching full `UserDetails` from the database on every request — including `enabled`, `accountNonLocked` flags — but nothing downstream actually read those fields before granting access. So the DB lookup was happening, but its result wasn't being used for anything except the username comparison." **Result:** "I added the three account-status checks into `isTokenValid`, plus a test that issues a token for an active user, disables that account afterward, and asserts the same still-unexpired token is now rejected. That closed a real gap: an admin disabling a compromised account wouldn't actually cut off that account's access until the token naturally expired."

This works for Amazon's "Dive Deep" and "Ownership" leadership principles almost unmodified — you found something by reading your own code critically, not because a test failed or someone told you to.

Part 5 — Likely follow-up questions and how to answer them

"How would you scale this to multiple instances?" > "Access token validation is already stateless — any instance can verify > any token with the shared secret, no session affinity needed. Refresh > tokens do need a shared datastore, which they already have — the > `refresh_token` table — so as long as all instances point at the same > database, this already horizontally scales without changes."

"What would you change for a true microservices setup?" > "Move from a shared HMAC secret to asymmetric signing — RS256. Each > issuing service holds a private key; every service that only *validates* > tokens holds just the public key. That way a compromised or misconfigured > downstream service can never forge tokens, only verify them — a much > smaller blast radius than every service holding a key capable of both > signing and forging."

"Why disable CSRF?" > "CSRF exploits the browser automatically attaching cookies to a request. > A Bearer token in an Authorization header isn't automatically attached by > the browser, so the attack vector CSRF protection defends against doesn't > apply here — it's a stateless, header-based API, not cookie-based > session auth. I'd bring CSRF protection back immediately if I ever added > cookie-based auth alongside this."

"Why BCrypt and not something else?" > "BCrypt is adaptive — you can tune its work factor as hardware gets > faster, unlike a fixed-cost hash like plain SHA-256. It's also > purpose-built for passwords: salted automatically, resistant to rainbow > tables, and it's what Spring Security ships as the default for exactly > this reason."

"What did you deliberately leave out of scope?" Naming real gaps unprompted signals seniority — don't wait to be caught out: > "No rate limiting on the login endpoint, so it's currently open to brute > force. No email verification step, so registration issues real access > immediately — in a system with any KYC requirement, like the ones I work > with in payments, I'd gate that behind verification first."

Part 6 — How to deliver this well (not just what to say)

- **Lead with the design decision, not the code.** Interviewers remember “access tokens are stateless, refresh tokens are revocable, here’s why” far better than a line-by-line walkthrough of `JwtService`.
- **Narrate tradeoffs unprompted.** Every “why X” answer above ends with a cost, not just a benefit. That’s what separates “I followed a tutorial” from “I understand the mechanism.”
- **Use the bug-you-found story once, well** — don’t manufacture more than one polished “I caught this myself” moment per interview; two or three feels rehearsed, one lands as genuine.
- **If you don’t know something, say so and reason toward it.** “I haven’t implemented RS256 here, but I’d expect the tradeoff to be X” is a strong answer. Guessing confidently and being wrong is a weak one.
- **Keep answers to about 60-90 seconds** unless asked to go deeper. Long unprompted monologues read as nervousness, not thoroughness — pause and let them redirect you.
- **On a whiteboard**, draw the two diagrams from this session in this order: the four-stage pipeline first (sets the frame), then the four validation checks with the branch at the end (shows depth). Don’t try to draw the full class diagram — it adds nothing your verbal explanation doesn’t already cover, and diagrams that are too dense read as anxious over-preparation rather than clarity.

Part 7 — 60-second closing line, if asked “anything else you’d add?”

“If this were headed to production, the next three things I’d prioritize are rate limiting on login, moving to RS256 for any multi-service deployment, and a logout endpoint that revokes all of a user’s refresh tokens at once, not just the one used in the current request.”

Ending on forward-looking, prioritized next steps — not “it’s basically done” — is what a senior answer sounds like.